Trees and random forests

Hicham Janati · *INSEA* · hjanati@insea.ac.ma

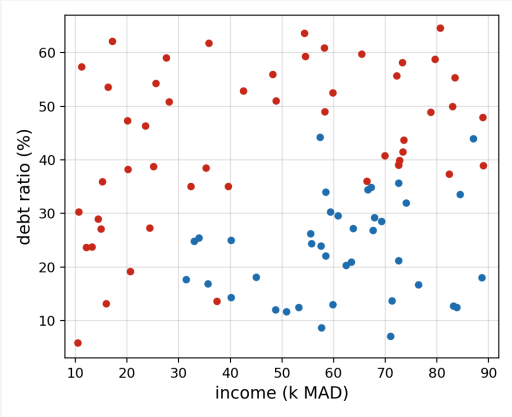
FSML · based on chapter 7 of the lecture notes

Decision trees

Predicting default

A bank wants to predict a future **loan default** (when a client does not reimburse) from two features.

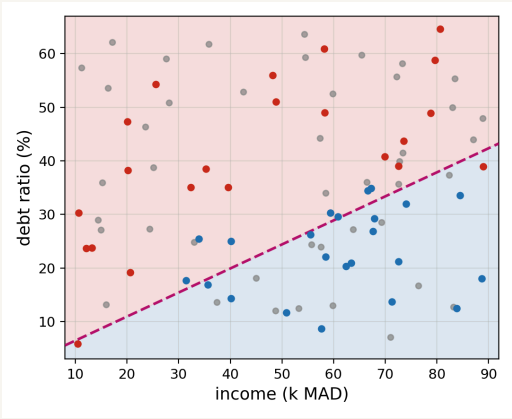
income (kDh)	debt (%)	default
80	59	1
33	25	0
58	49	1
72	56	1
⋮	⋮	⋮
62	29	?



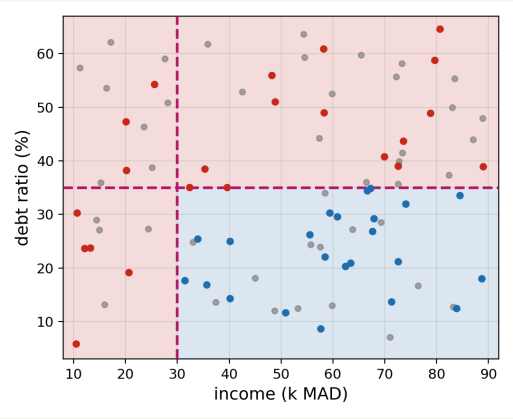
Predicting default

A bank wants to predict a future **loan default** (when a client does not reimburse) from two features.

income (kDh)	debt (%)	default
80	59	1
33	25	0
58	49	1
72	56	1
⋮	⋮	⋮
62	29	?



linear model: train **20%**, test **16%**



best function: train **0%**, test **9%**

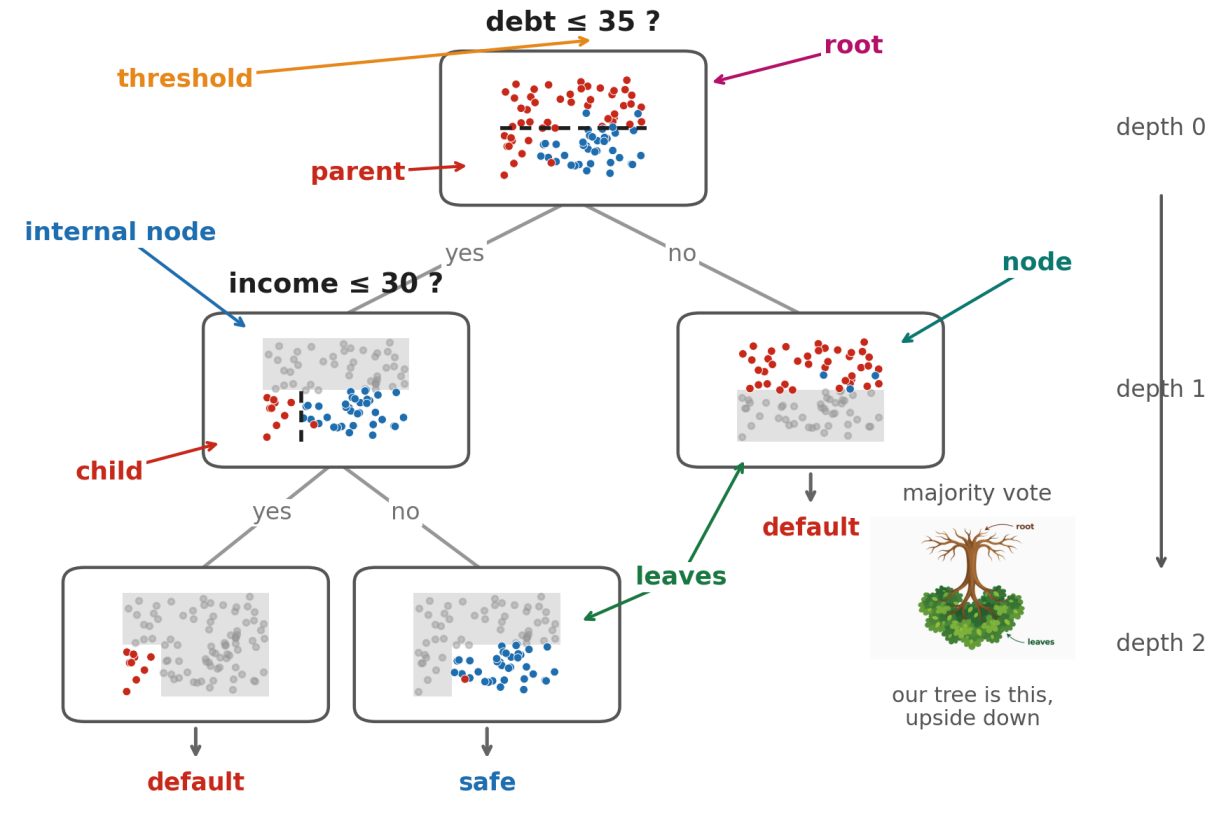
How can we define such a function ?

$$f(x) = 1 \Leftrightarrow \text{income} \leq 30 \text{ or debt } \geq 35$$

$$x_1 \leq 30 \text{ or } x_2 \geq 35$$

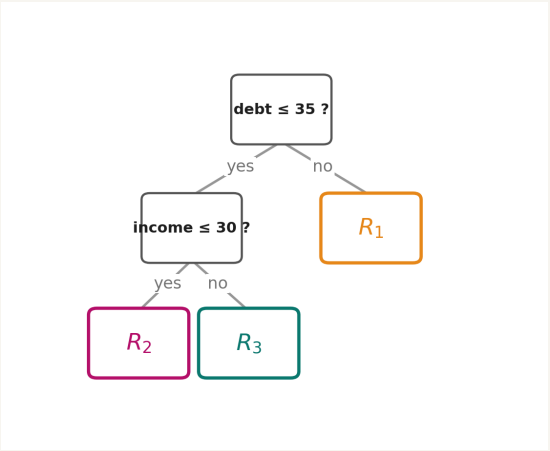
By thresholding the feature space: **a decision tree**.

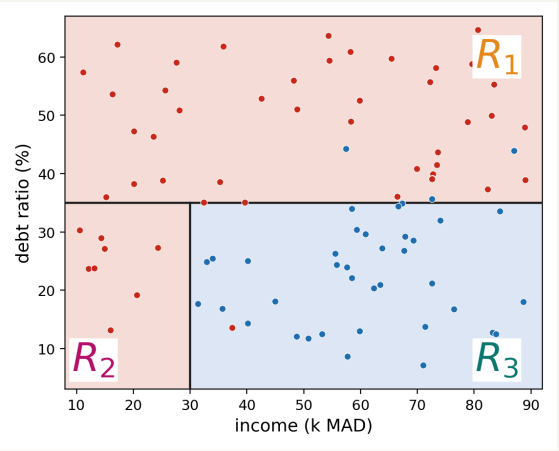
Visualizing a classification tree



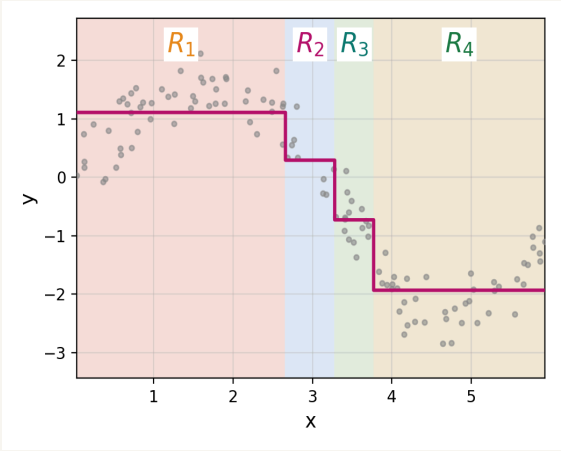
Decision trees: piecewise-constant functions

A decision tree carves the space into a **partition of regions**: its **leaves**.





Classification



Regression

On each region R_j: what is the value of f ?

Classification: the region's majority class.

Regression: the region's average, $\mathbf{x} \in R_j \Rightarrow f(\mathbf{x}) = \bar{y}_{R_j}$

The best regression tree, formally

What is the **best regression tree** for a partition of J regions ?

$$\min_{R_1,\dots,R_J} \sum_{j=1}^{J} \sum_{i:\, \mathbf{x}_i \in R_j} \left(y_i - \bar{y}_{R_j}\right)^2$$

The unknown is the **partition itself**, combinatorially many candidates. NP-hard problem.



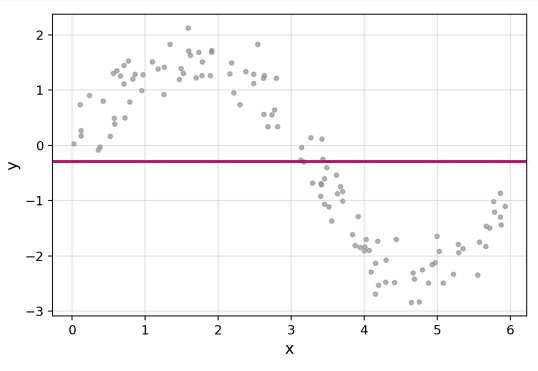
Greedy optimization (Algorithme *glouton*):

0. Start with root node = $\mathbb{R}^d$
1. find the best split at the *current* node
2. apply it to get two **children** nodes
3. repeat steps 1 and 2 on each child node

CART algorithm

Regression trees

What about **regression**: data $(\mathbf{x}_i, y_i)$ with $y_i \in \mathbb{R}$. What does a **tree** f look like ?



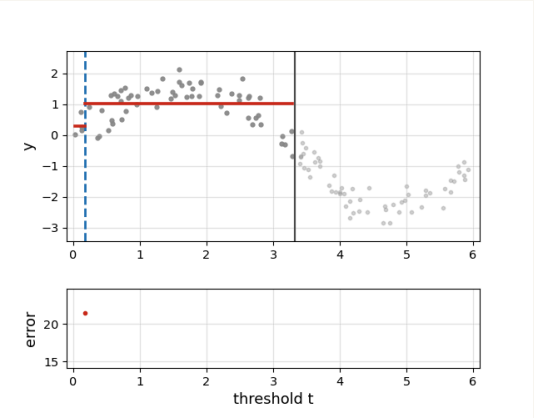
Start simple: fix **depth = 0**: no split allowed. What is the *best* possible tree ?

A **constant** ! One leaf, predicting the global average $\bar{y}$

Depth 1: one split allowed. We only have one variable x. **Where** do we cut it ?

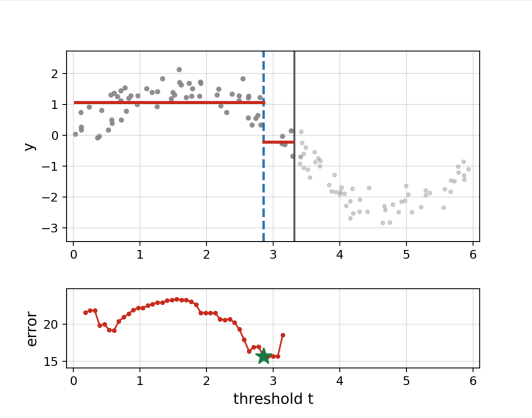
The first split, live

Recurse: left, then right

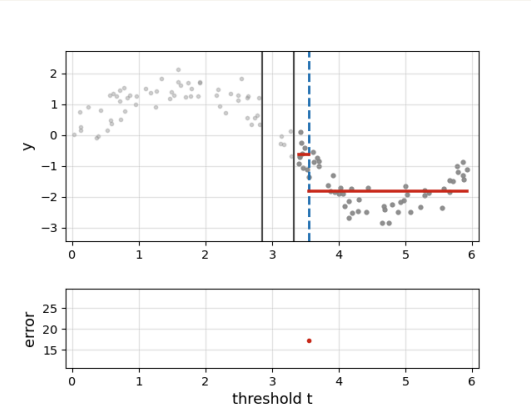


the same sweep, **inside the left** region

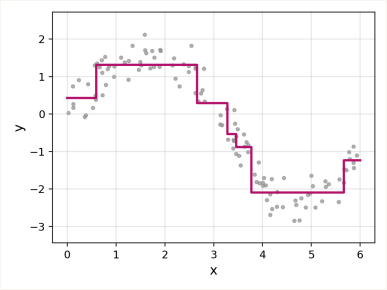
Recurse: left, then right



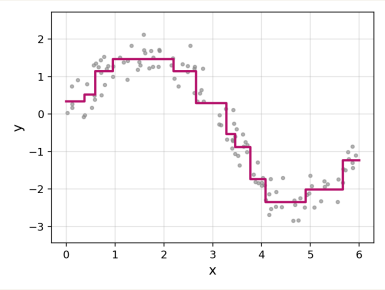
the same sweep, **inside the left** region



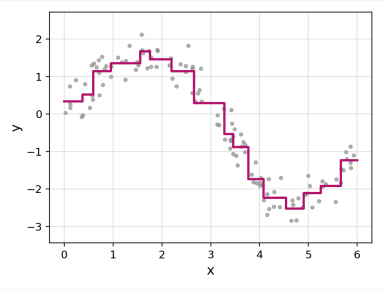
then **inside the right**



... and again: **depth 3**



depth 5



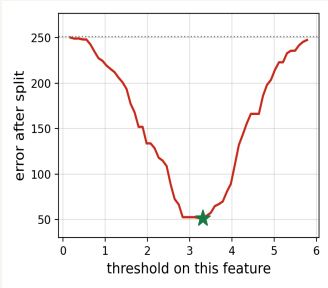
depth 10

Several features ?

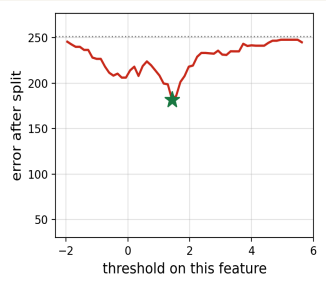
That was one variable. With d features: **which one** gets the cut ?

CART is *greedy*: THE BEST ONE in terms of **decreasing prediction error**.

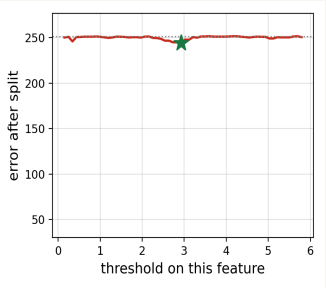
So we need to try to split **every single** feature:



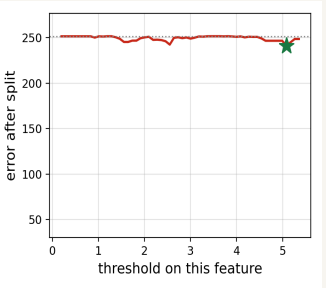
feature 1



feature 2



feature 3



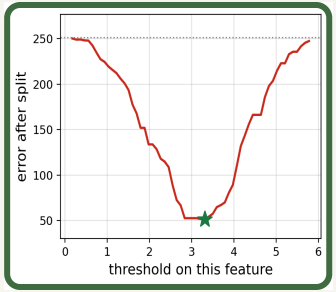
feature 4

Several features ?

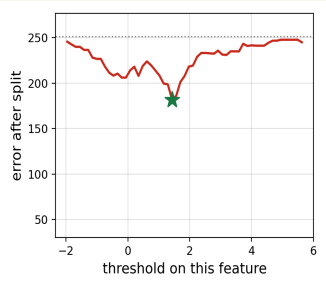
That was one variable. With d features: **which one** gets the cut ?

CART is *greedy*: THE BEST ONE in terms of **decreasing prediction error**.

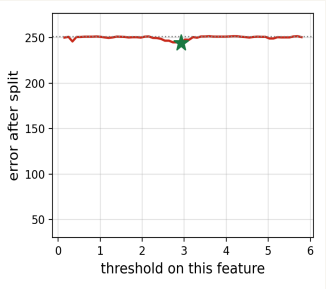
So we need to try to split **every single** feature:



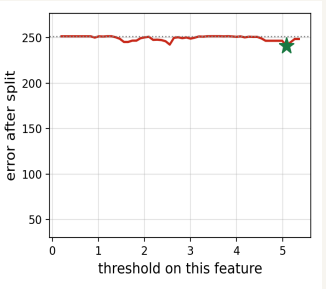
feature 1



feature 2



feature 3

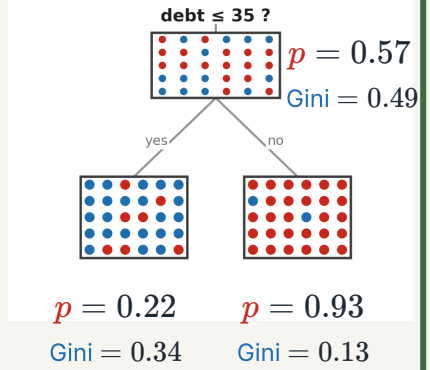


feature 4

Find the best **threshold** for every **feature**, select the (feature, threshold) pair with the **largest gain**.

Scoring a classification split

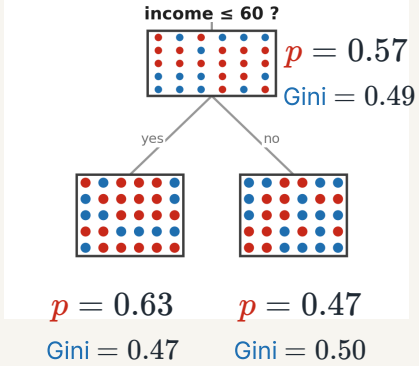
Back to classification: the **same parent node**, two candidate cut:



split A

average Gini ≈ 0.24

Δ Gini $= -0.25$



split B

average Gini ≈ 0.48

Δ Gini $= -0.01$

Which split is better ?

Split A: its children are *cleaner*.

How can we define a score for *cleanliness* ?

p = the frequency of class 1 in a leaf node

$$\mathbb{P}(y_1 \neq y_2) = 2p(1-p)$$

Gini Impurity

If we draw samples at random $\mathbb{P}(y_1 \neq y_2)$ must be low.

CART, unified

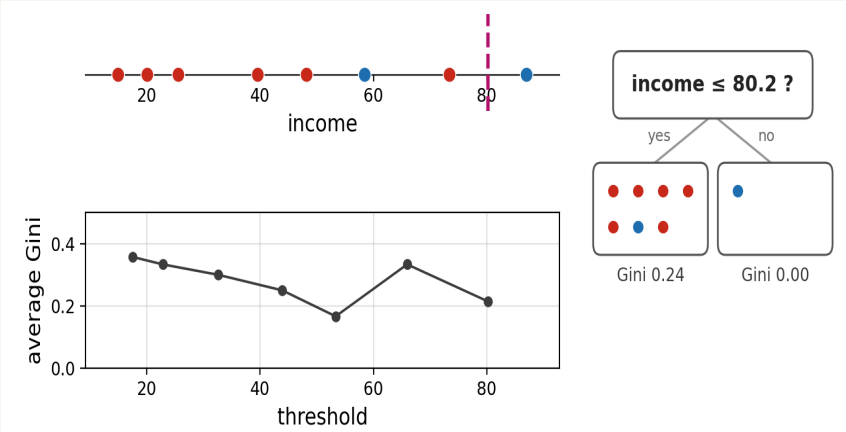
The same greedy recursion runs both classification and regression trees:

	regression	classification
split score	children's squared error	children's Gini $2p(1-p)$
leaf output	the leaf's average $\bar{y}$	majority class / proportions
the recursion	scan every (feature, threshold), keep the largest gain, repeat on the children	

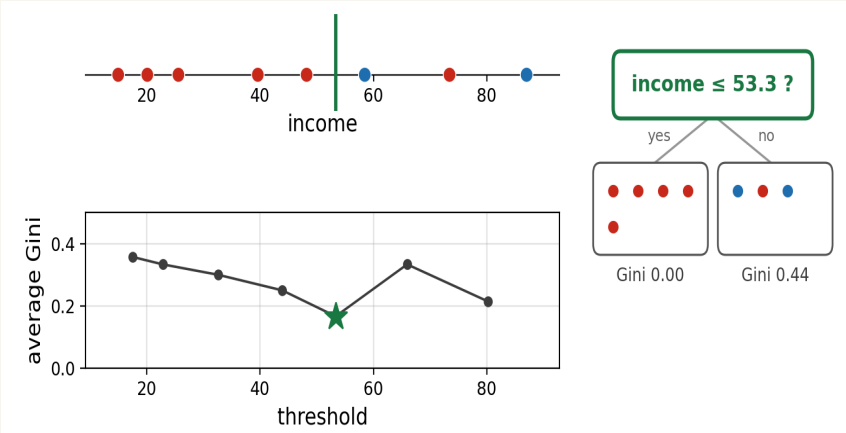
CART algorithm: *Classification And Regresion Trees* (1984)

Trees in practice

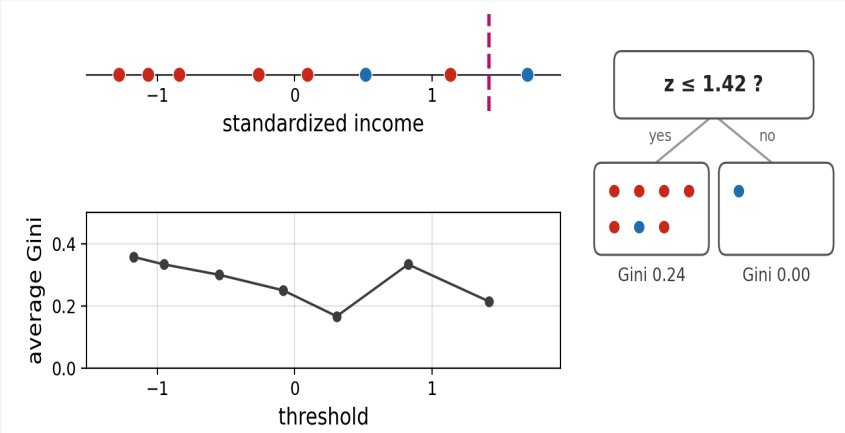
Rescaling a feature ?



Rescaling a feature ?

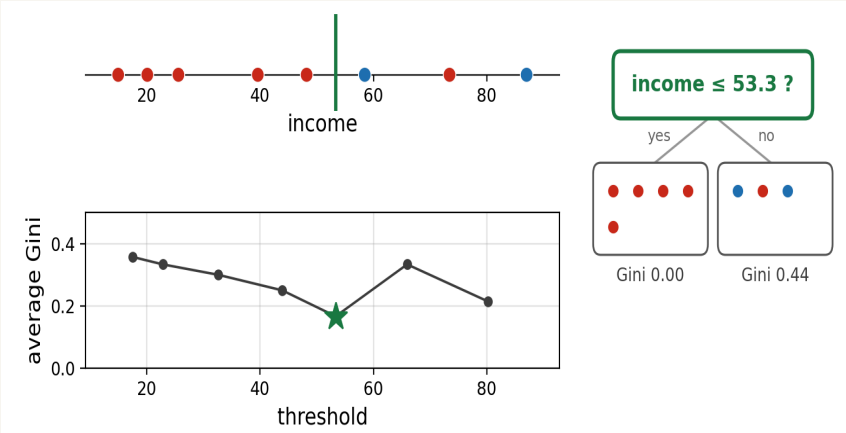


best split: $\text{income} \leq 53.3$

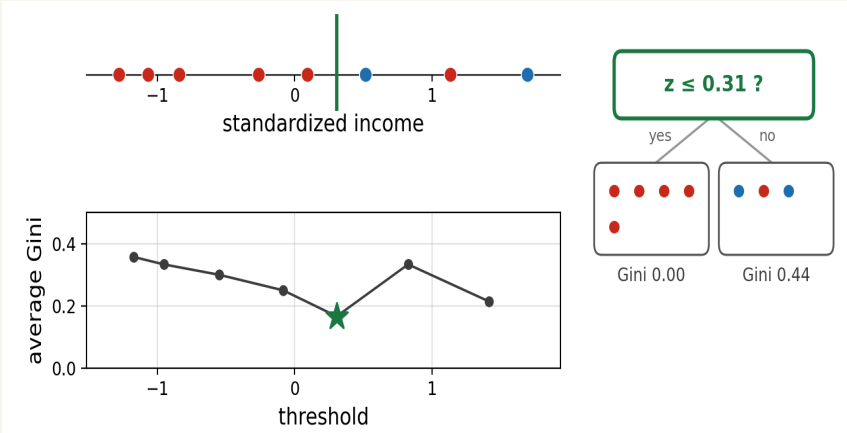


What happens to a **tree** on standardized income ?

Rescaling a feature ?



best split: $\text{income} \leq 53.3$



same scores, $z \leq 0.31$: the **same clients** on each side

What happens to a **tree** on standardized income ?

Same order, same splits, **same tree**: a tree needs no standardization.

What if we apply $\log$ to income ?

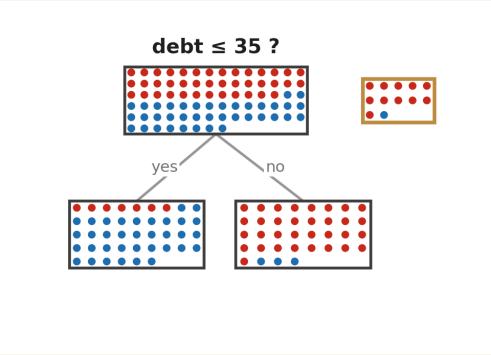
Same tree !

Missing values ?

Some clients left their **debt** blank. The root asks $\text{debt} \leq 35$: where do **they** go ?

income (kDh)	debt (%)	default
80	59	1
33	25	0
58	?	1
72	?	1
⋮	⋮	⋮

- debt missing: 12 of 90 clients

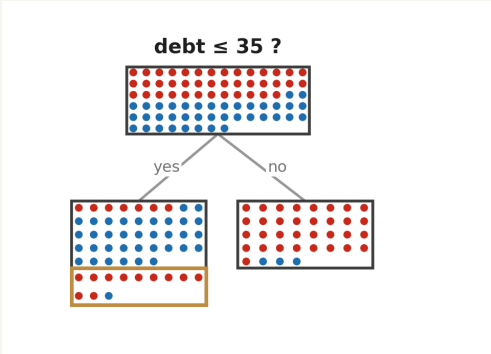


Missing values ?

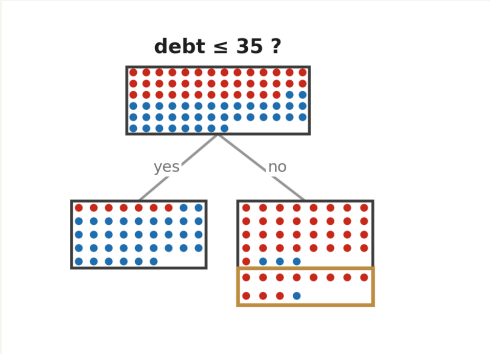
Some clients left their **debt** blank. The root asks $\text{debt} \leq 35$: where do **they** go ?

income (kDh)	debt (%)	default
80	59	1
33	25	0
58	?	1
72	?	1
⋮	⋮	⋮

- debt missing: 12 of 90 clients



missing → **left**: average Gini **0.33**

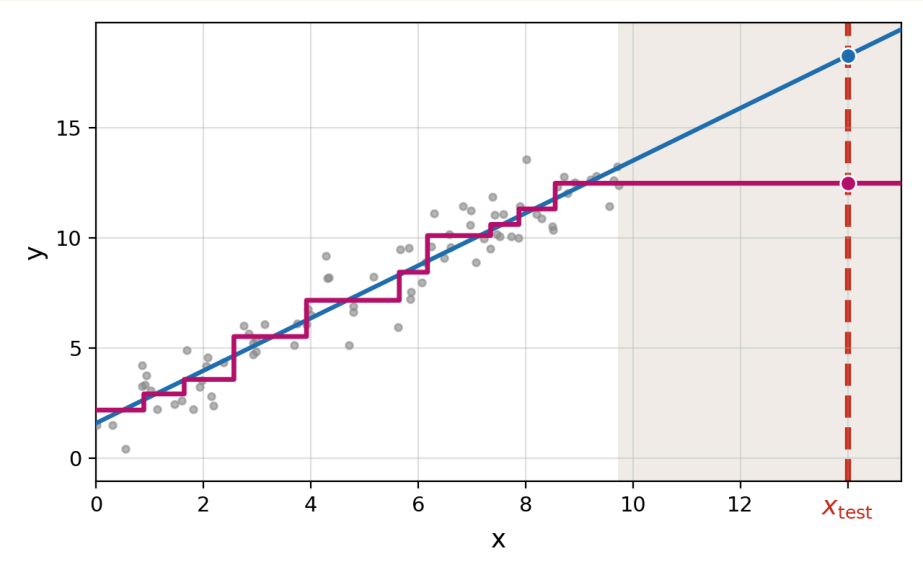


missing → **right**: average Gini **0.21**

The lower-loss side becomes the split's **default direction**, stored with it: a new client with a missing debt follows it.

No imputation: scikit-learn trees and random forests accept `NaN` directly, and boosting libraries do the same.

Extrapolation ?



training data up to $x = 9.7$; the shaded region holds **no data**

A new point $x_{\text{test}} = 14$ (**red line**): what do a **line** and a **tree** predict ?

line: 18.3, it follows the trend

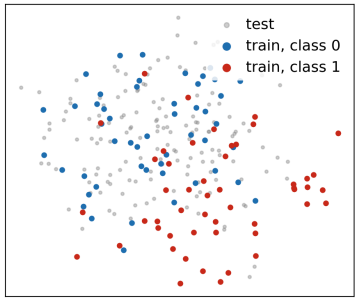
tree: 12.5, exactly its value at $x = 9.7$

Past the last training value, a tree **freezes** at its boundary leaf's constant; a line keeps moving.

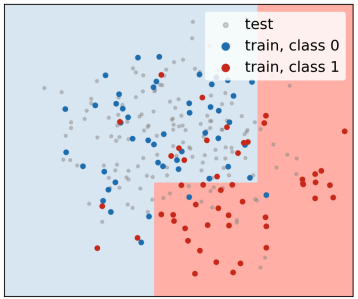
Complexity of a decision tree

Depth

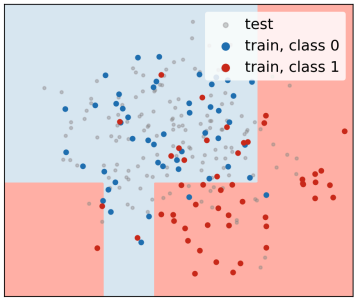
Can a tree overfit ?



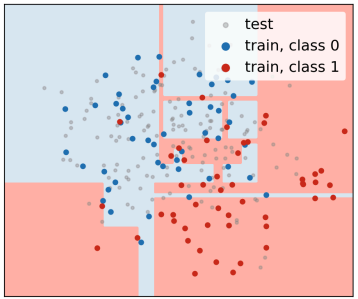
the data



depth 2



depth 3



depth 10

train / **test** error

0.17 / **0.23**

0.15 / **0.24**

0.01 / **0.28**

What happens if when we increase depth further ?

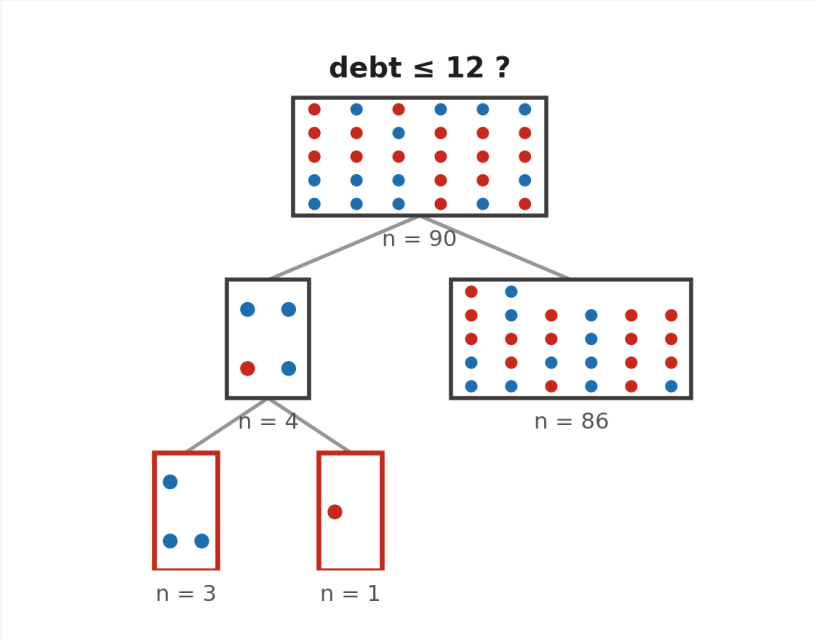
We keep splitting until **all leaves** are pure !

Depth is not enough

So we should **restrict max depth** to avoid overfitting.

What about **this** depth-2 tree ?

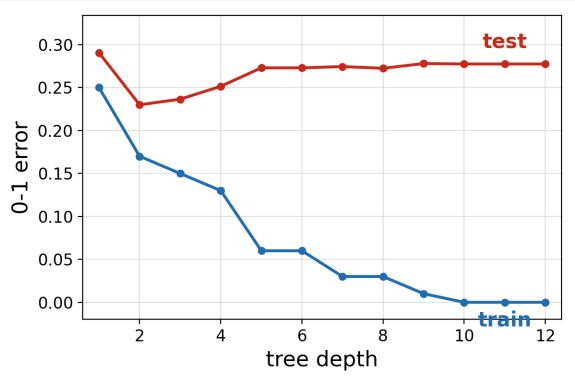
We need to avoid having very small leaves.

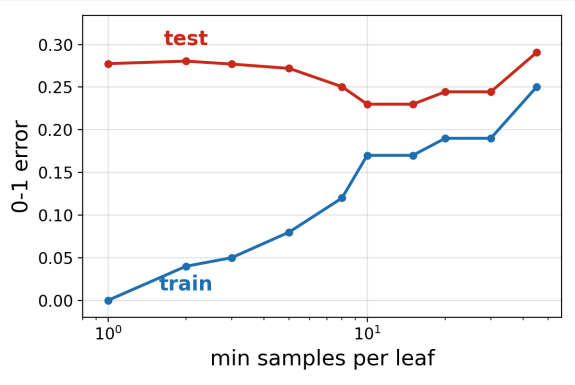


Another hyperparameter to restrict the possible splits: *minimum samples per leaf*

U-curve

What happens to the **train error** when we increase each hyperparameter ?





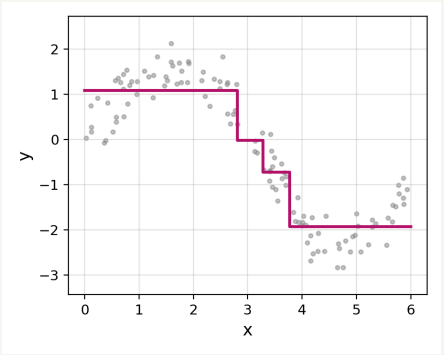
And the **test error** ?

These are (two of) the main hyperparameters of a Decision Tree

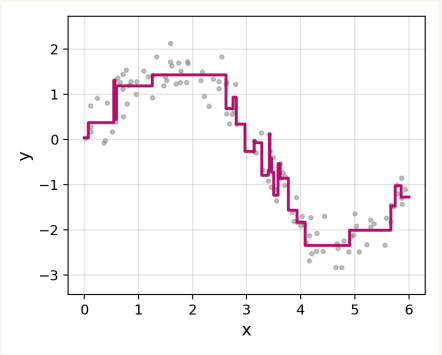
Chosen exactly as in chapter 5: a **grid**, **cross-validation**.

Trees have high variance

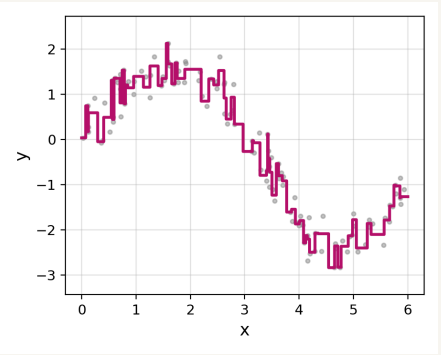
To fit anything interesting, a tree must be **deep**. What is the price ?



depth 2: train 0.23, test 0.29



depth 5: train 0.07, test 0.17



depth 12: train **0.03**, test **0.22**

The **useful** trees are the deep ones, and deep trees **overfit**: they have **high variance**.

Bagging and random forests

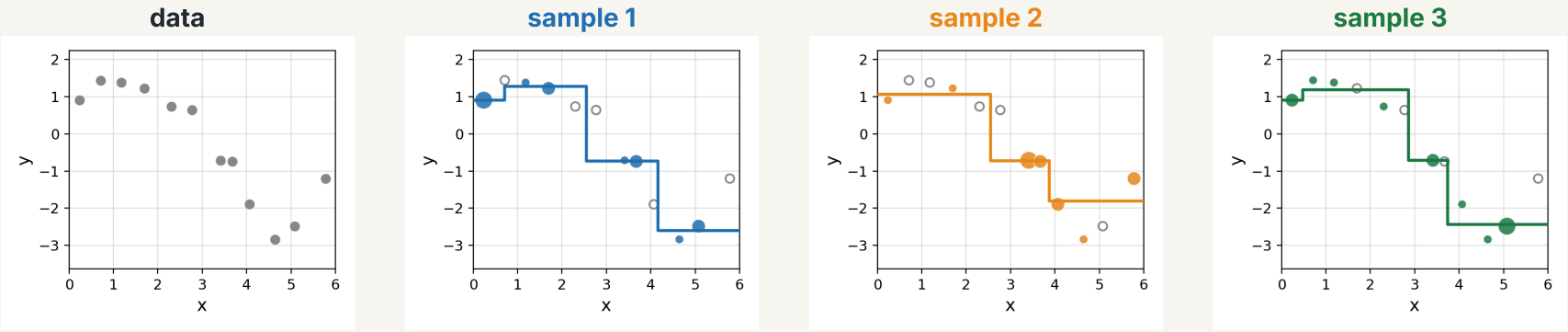
Bagging: the idea

We want to **reduce the variance**. How do we reduce the variance of a random variable ?

Average different estimates: $\mathbb{V}(\bar{X}) = \sigma^2/n$

But average **what** ? We only have **one** dataset, **one** tree.

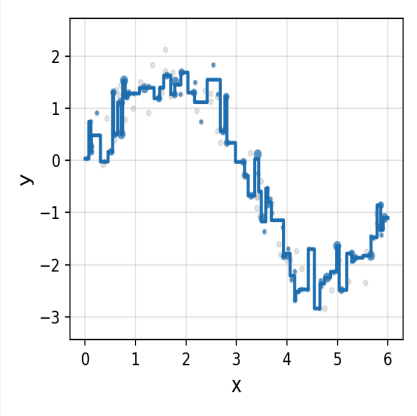
The **bootstrap**: resample the rows **with replacement**, fit one tree per replica.



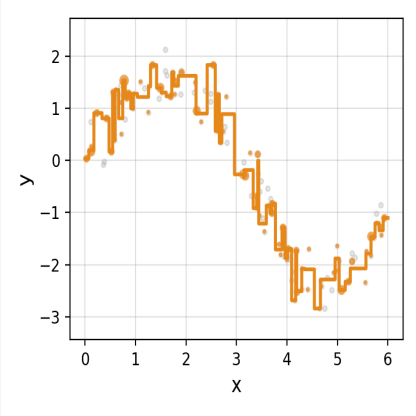
hollow dots: rows left out of the sample; big dots: rows drawn several times

Bagging: one tree per sample

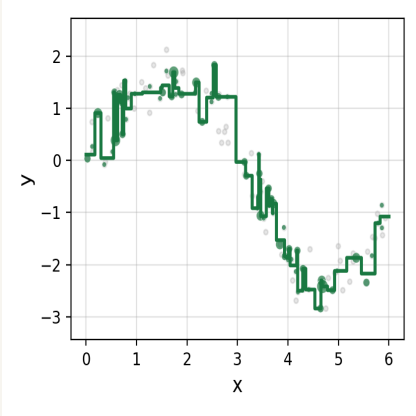
Same with all $n = 120$ rows: each bootstrap sample grows its own **deep** tree.



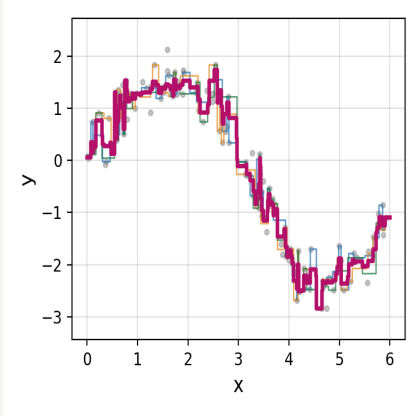
tree 1: test MSE 0.21



tree 2: test MSE 0.25



tree 3: test MSE 0.25



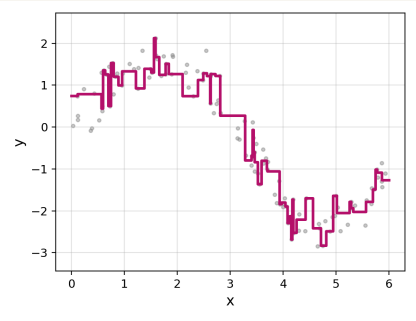
average: test MSE 0.20

big dots: rows drawn several times; faint dots: rows not drawn

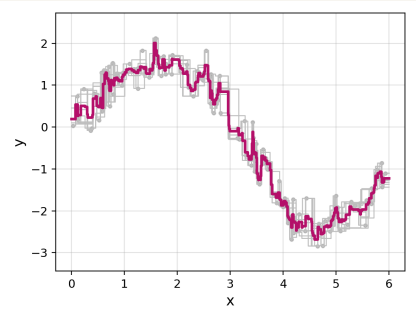
The average beats each of its trees. What if we average **many** trees ?

Bagging: Bootstrap Aggregation

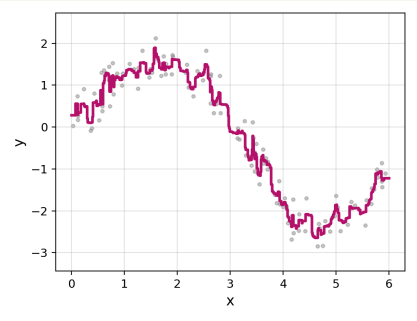
Draw n rows with replacement, fit one **deep** tree; repeat B times; **average** the B predictions.



one bootstrap tree ($B = 1$)



$B = 12$ **trees** (gray) and their **average**



the average of $B = 200$ trees

Even with B = 200, the variance is still high, why could that be?

Random forests: decorrelate

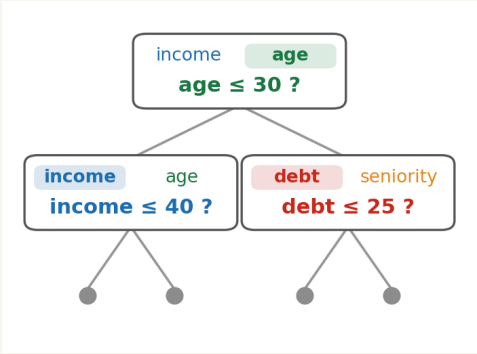
Random forests: decorrelate

The forest, one tree at a time

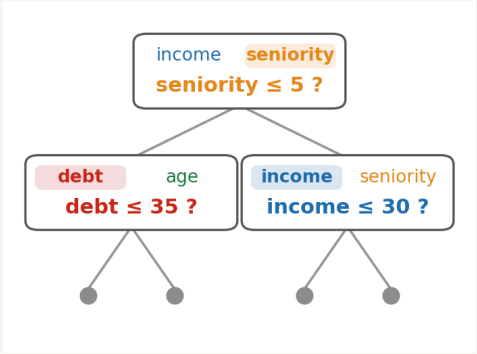
At **every split**, each node considers a random subset of $m < d$ features to split.

data (n rows, features **income**, **debt**, **age**, **seniority**)

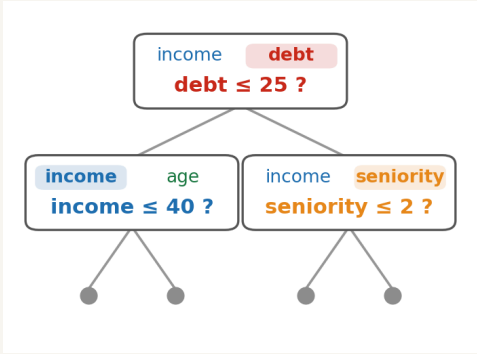
bootstrap 1 → T_1



bootstrap 2 → T_2



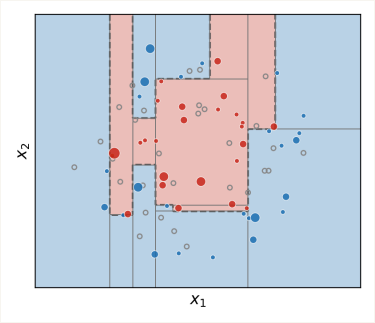
bootstrap 3 → T_3



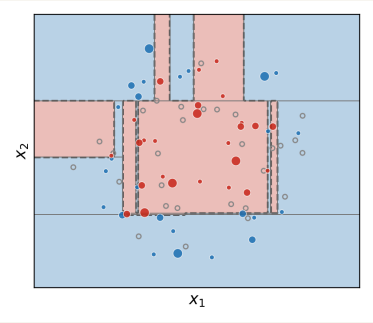
different random subsets → **different splits** → decorrelated trees

then **average**: $\hat{F}_B(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})$. That is a **random regression forest**.

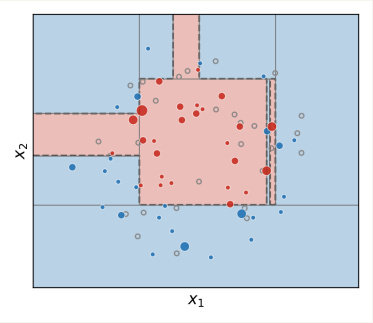
Combining classification trees



tree 1



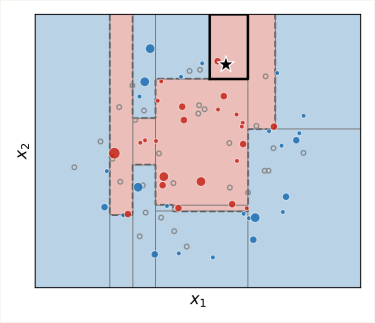
tree 2



tree 3

How do we "average" classification trees ?

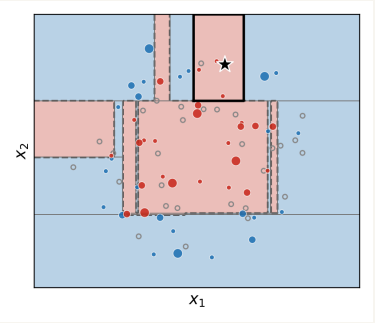
Combining classification trees



tree 1

$\hat{p}_1(\mathbf{x}) = 1$

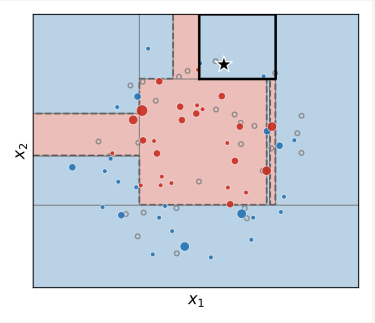
test error 0.28



tree 2

$\hat{p}_2(\mathbf{x}) = 1$

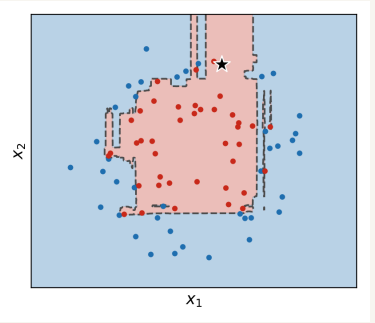
test error 0.24



tree 3

$\hat{p}_3(\mathbf{x}) = 0$

test error 0.23



B = 200 trees

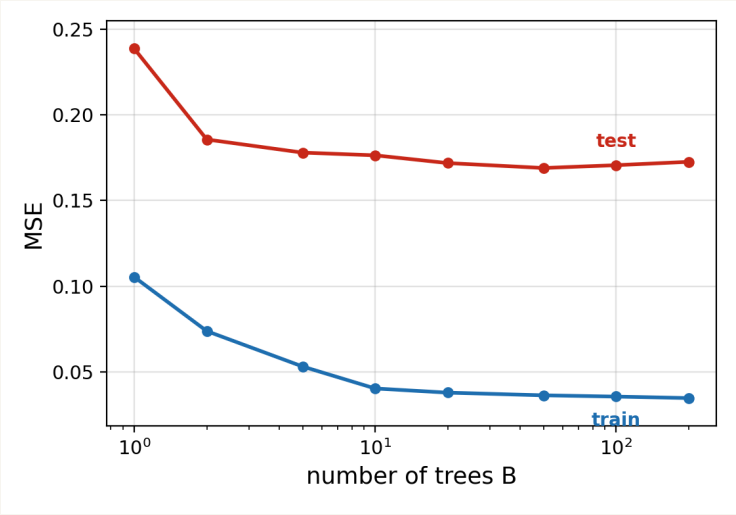
$\hat{p}(\mathbf{x}) = 0.74$

test error **0.21**

How do we "average" classification trees ?

Average the frequencies: $\hat{p}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B}\hat{p}_b(\mathbf{x})$

Number of trees



What should be the effect of a large B on the **train** error ?

And on the **test** error ?

The error falls and **flattens**. B is not a complexity dial; more trees reduce overfitting.

How do we choose B ?

No cross-validation needed: take B large enough for the error to flatten; more trees only cost computing time.

Exercise 1
Out-of-bag error

A bootstrap sample draws n rows **with replacement** out of the n rows of the data, and a forest grows B trees, one per bootstrap sample.

1. What is the probability that row i is **not** drawn in a given bootstrap sample ? What is its limit when $n \to +\infty$?

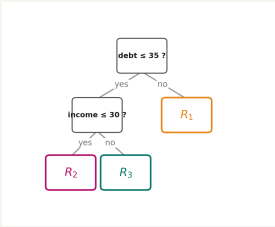
2. Consider the number of trees Z $(< B)$ trees that never saw row i. How can we model its distribution ? What is the expected number of trees Z that never saw row i ?

3. Using only those trees, propose an estimate of the **test** error without any validation set.

Each tree never sees $\approx 37\%$ of the rows: predicting every row with those trees gives a test error estimate **for free**.

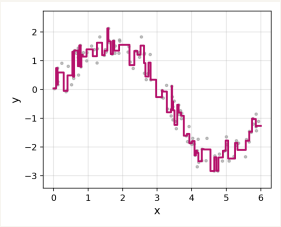
What to remember

a tree



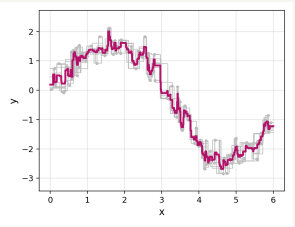
greedy splits: a piecewise-constant function

its complexity



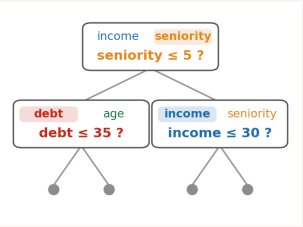
a deep tree overfits: **high variance**

bagging



average **B** bootstrap trees: the variance falls

random forest



random features at every split: **decorrelated** trees

Deep trees have **low bias** and **high variance**: averaging decorrelated trees decreases the variance

Out-of-bag error: each tree misses $\approx 37\%$ of the rows, which give a test error estimate **for free**.

Contents

1. Decision trees
2. CART algorithm
3. Trees in practice
4. Complexity of a decision tree
5. Bagging and random forests